



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3106-01 - APPLICATION DEVELOPMENT (WBL)

TOPIC:

Restaurant Order Waiter System

PREPARED BY :

NAME	MATRIC NO
AIN NURNABILA BINTI MOHD AZHAR	A23CS0207

PREPARED FOR:

DR. AHMAD NAJMI BIN AMERHAIDER NUAR

TABLE OF CONTENTS

1. Introduction	2
2. Software, Language, Tools, API and Templates	4
3. Development Steps	6
4. System Interface.....	10

1. Introduction

The Restaurant Waiter Order System is a mobile application built with Flutter and Dart. It is designed to give restaurant waiters a straightforward digital tool for managing the menu and recording customer orders. Rather than writing orders by hand or relying on verbal communication with the kitchen, the waiter can use this app to handle everything from a mobile device in real time.

The application has no login or registration screen. This is a deliberate design choice based on the project requirements. The app is intended for internal use by waiters who are already on the restaurant floor, so it opens directly into the main interface without any authentication step. This keeps the workflow fast and removes unnecessary friction.

The core purpose of the app is to demonstrate full CRUD functionality connected to a live cloud database. CRUD stands for Create, Read, Update, and Delete, which are the four fundamental operations any data-driven application must support. In this app, CRUD is applied to two main areas: the restaurant menu and customer orders. The waiter can add, view, edit, and delete menu items, and create, view, update, and cancel orders throughout the dining session.

The data is organized into three tables in a Supabase PostgreSQL database. The first table, `menu_items`, stores all dishes and drinks with their names, prices, categories, and availability status. The second table, `orders`, records each order with a table number, status, total price, and the time it was created. The third table, `order_items`, links orders to their specific dishes and captures the name and price of each item at the time the order was placed. This last point is particularly important because it means historical orders remain accurate even if menu prices change later.

Every order follows a fixed lifecycle with four stages. It starts as Pending when the waiter takes the order and ends as Paid when the customer settles the bill. The two stages in between are Preparing, when the kitchen is cooking the food, and served, when the food has been delivered to the table. The waiter controls these transitions from the order detail screen. Cancelling an order is only possible while it is still in the Pending stage.

This project was completed as an individual test submission for the SECP3106-01 Application Development WBL course at Universiti Teknologi Malaysia. It demonstrates the ability to design and implement a multi-screen Flutter application with real backend

connectivity using Supabase, following the complete CRUD requirements set out in the course brief.

2. Software, Language, Tools, API and Templates

This section provides a full list of all technologies and tools used to develop the Restaurant Waiter Order System. The choices were guided by the course requirements and by the practical needs of building a mobile app with a live cloud database.

Category	Tool / Technology	Purpose in This Project
Language	Dart	Primary programming language for all application logic and UI code.
Framework	Flutter (latest stable)	Cross-platform mobile framework that builds a native Android app from a single codebase.
IDE	Visual Studio Code	Code editor used throughout development with the Flutter and Dart extensions installed.
Emulator	Android Emulator (Android Studio)	Used to run and test the app on a virtual Android device during development.
Backend	Supabase (PostgreSQL + RLS)	Cloud database service providing a real PostgreSQL database for persistent data storage.
Backend SDK	supabase_flutter	Official Flutter package that handles all communication between the app and Supabase.
State Management	setState	Built-in Flutter mechanism to rebuild widgets when data changes, sufficient for this app.
UI Widgets	Material 3 (Flutter built-in)	Material Design 3 components including Scaffold, ListView, Card, TextFormField, AlertDialog, and NavigationBar.

Version Control	Git and GitHub	Used to track code changes and keep a backup of the project throughout development.
-----------------	----------------	---

Supabase was chosen as the backend because it offers a genuine PostgreSQL relational database that supports proper table relationships, foreign keys, and Row Level Security. Unlike local storage solutions such as shared preferences or SQLite, Supabase stores data in the cloud. This means records survive an app restart and can be accessed from any device, which satisfies the requirement for a real cloud backend connection.

Flutter was used because it provides a rich set of Material 3 widgets out of the box and compiles a native Android app from a single Dart codebase. The setState mechanism was sufficient for managing state in this app because the data flow is straightforward: the screen loads data from Supabase when it appears and refreshes that data after any operation completes. No additional state management library was needed.

3. Development Steps

This section documents the actual steps taken to build the application. Each step includes a short note explaining what was done and the reasoning behind key decisions.

Step 1: Create the Flutter Project

The project was initialized using the command `flutter create resto_order` in the VS Code integrated terminal. Once the project was ready, it was opened in VS Code and launched on the Android emulator with `flutter run` to verify that the base setup was working before making any changes. The default counter app appeared on the emulator without errors, confirming that the development environment was fully operational.

Step 2: Set Up the Supabase Project

A new Supabase project was created at supabase.com and named `resto_order`. The server region was set to Southeast Asia (Singapore) because this gives the lowest network latency for users in Malaysia. After the project finished provisioning, two pieces of information were copied from the Project Settings page: the Project URL and the anon public key. These credentials are required to connect the Flutter app to the Supabase backend.

Step 3: Add the Supabase Package and Initialize the Connection

The `supabase_flutter` package was added by running `flutter pub add supabase_flutter` to the terminal. This command updated the `pubspec.yaml` file and fetched the package automatically. The `main.dart` file was then updated so that `Supabase.initialize()` is called with the project URL and anon key before `runApp()` executes. The line `WidgetsFlutterBinding.ensureInitialized()` was placed at the top of the `main()` function to ensure the Flutter engine was fully ready before any asynchronous initialisation took place.

Step 4: Create the Database Schema

Three tables were created in Supabase using the SQL Editor. The `menu_items` table stores each menu entry with a UUID primary key, name, price, category, and a Boolean available field. The `orders` table records each order session with a table number, a status field starting at Pending, a calculated total, and a `created_at` timestamp. The `order_items` table bridges orders and menu items with a foreign key to each, plus `name_snapshot` and `price_snapshot` columns.

The snapshot columns copy the name and price from the menu item at the moment the order is created. This design decision ensures that editing or deleting a menu item later does not alter any existing order records. Row Level Security was enabled on all three tables, and an open policy was applied to allow all read and write operations, which is appropriate for a test environment without user authentication.

Step 5: Set Up the Folder Structure and Data Models

Before writing screen code, the lib folder was organised into three subdirectories: models, screens, and services. The model's folder contains three Dart classes, one for each table. Each class has a fromMap() constructor that converts a Supabase response map into a Dart object, and a toMap() method that converts the object back into a map for database insertion. This separation means the screen code never deals with raw map data directly, which reduces errors and makes the code easier to read.

Step 6: Build the Menu Tab

The Menu tab consists of two screens. The menu list screen loads all menu items from Supabase when it initializes and displays each one in a Card widget. The card shows the item name, category, price, a Switch for the available toggle, and icon buttons for editing and deleting. Pressing delete shows an AlertDialog asking for confirmation before the record is removed. After any operation, the screen reloads its data from Supabase to keep the display current.

The add and edit screen is the same widget used in two modes. When no MenuItem is passed in, it presents an empty form for creating a new item. When an existing MenuItem is passed in, it pre-fills all fields with the current values. The form validates that the name field is not empty, and that the price field contains a valid decimal number before saving. A DropdownButtonFormField is used for the category to prevent free-text input errors.

Step 7: Build the Orders Tab

The Orders tab shows a list of all orders retrieved from Supabase, sorted with the newest at the top. A row of FilterChip widgets at the top of the screen lets the waiter narrow the list to a single status. Each order is shown in a Card with the table number, price, creation time, and a colour-coded status chip: orange for Pending, blue for Preparing, green for Served, and grey for Paid.

The New Order screen is accessed via the floating action button. It shows a table of number of input at the top and the full list of available menu items below, filtered so that only items marked as available appear. Each item has increment and decrement buttons for setting the quantity. A live total at the bottom of updates as items are added or removed. When Place Order is tapped, the service layer inserts one row into the orders table and one row per selected item into the order_items table, using the name and price snapshots at that point in time.

Step 8: Build the Order Detail Screen

The Order Detail screen fetches all order items for a given order and displays them in a list. Each row shows the item name from the snapshot, the price from the snapshot, the quantity, and the calculated line total. The overall order total is shown below the list. An action button with a context-sensitive label lets the waiter advance the order to the next stage. When the order reaches Paid, the advance button disappears. A Cancel Order button is shown only when the status is Pending, and pressing it triggers an AlertDialog before deleting the order and all its line items from the database.

Step 9: Testing on the Emulator

The full application flow was tested on the Android emulator to verify every operation. Several menu items were added across different categories. A new order was placed for Table 5 with three items from the menu. The order was advanced through all four stages from Pending to Paid. A second order was created and then cancelled while still in Pending to confirm that cancellation works and that the record is removed from the list. An existing menu item was edited to confirm that changes save correctly. Another item was deleted to confirm that the delete operation works. The app was restarted and all data was still present in Supabase, confirming that the cloud connection persists data correctly.

Step 10: Taking Screenshots

Screenshots of all five required screens were captured from the Android emulator using the camera icon in the emulator side toolbar. One screenshot was taken per screen, covering the menu list, the add and edit form, the orders list with filter chips, the new order screen with items and a total, and the order detail screen with the status advance button.

Step 11: Writing the Report and Submitting

The report was written in four sections as required by the brief. The project folder was zipped after removing the build directory to keep the file size manageable. The zip file and this report PDF were uploaded to eLearning under the file name specified in the submission instructions before the deadline of 18 July 2026 at 11:59 PM.

4. System Interface

This section presents screenshots of the five main screens in the Restaurant Waiter Order System. All screenshots were taken from the Android emulator during functional testing after the app was fully built. Each screenshot is followed by a short description of what the screen does and what it shows.

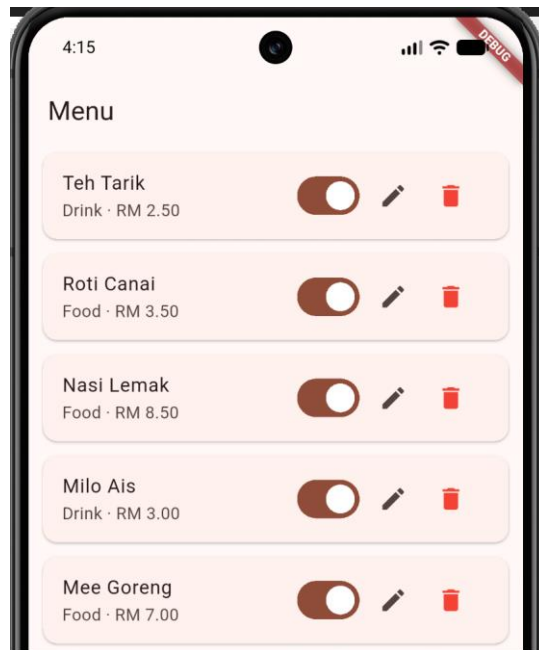


Figure 1: Menu List Screen

This screen shows the complete list of menu items stored in the Supabase database. Each item is displayed inside a Card widget showing the item name, category, and price on the left, and a Switch for toggling availability, an edit icon, and a delete icon on the right. The floating action button at the bottom right opens the Add Item form.

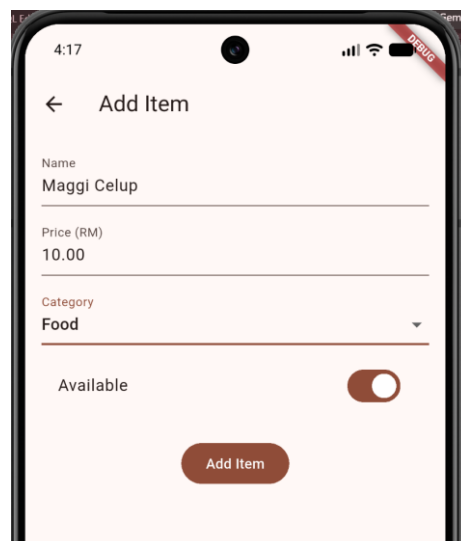


Figure 2: Add / Edit Menu Item Screen

This screen handles both adding a new menu item and editing an existing one. It contains a text field for the item name, a numeric field for the price, a dropdown to select the category from preset options, and a switch for the available status. The Save button validates all fields before writing to the database. When editing, all fields are pre-filled with the current values of the selected item.

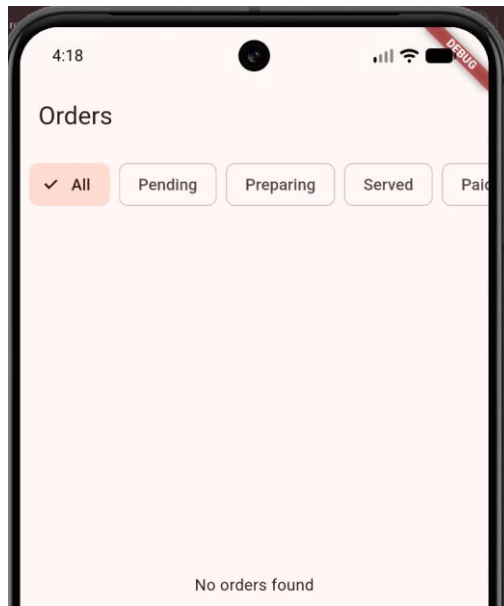


Figure 3: Orders List Screen

This screen displays all orders in the system, sorted with the most recent at the top. FilterChip widgets at the top allow the waiter to filter the list by order status: All, Pending, Preparing, Served, or Paid. Each order card shows the table number, total amount, creation timestamp, and a colour-coded chip indicating the current status. Tapping an order card opens the Order Detail screen.

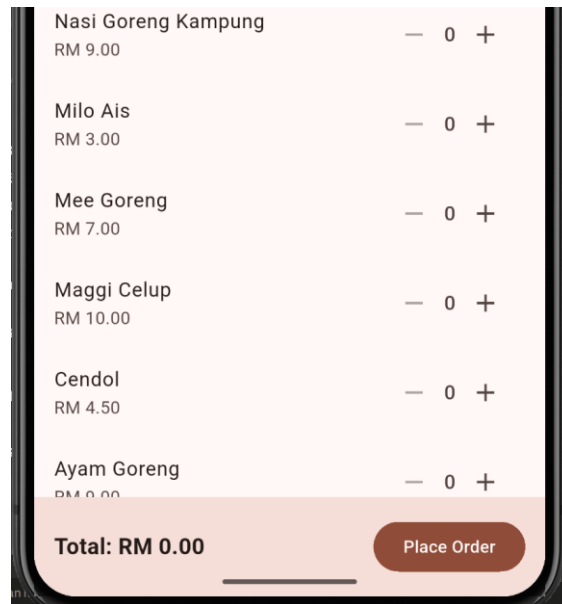


Figure 4: New Order Screen

This screen is used to create a new customer order. The waiter enters the table number at the top and then browses the list of available menu items below. Each item has plus and minus buttons to adjust the quantity. Items with zero quantity are not included in the order. The total price updates automatically at the bottom as items are added or removed. Tapping Place Order saves the order and all its line items to the database.

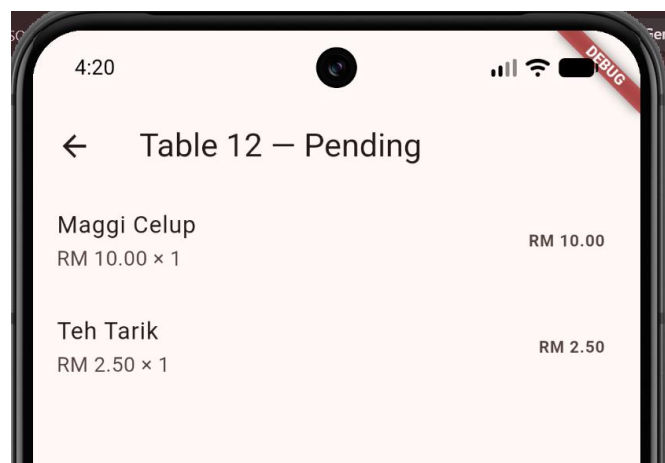


Figure 5: Order Detail Screen

This screen shows the full contents of a selected order. Each line item lists the name snapshot, price snapshot, quantity, and calculated line total. The overall order total is displayed at the bottom. An action button with a descriptive label advances the order to the next status stage. A red Cancel Order button appears only when the order is still in Pending status, allowing the waiter to remove the order from the system before it is sent to the kitchen.

REFERENCES

- Biørn-Hansen, A., Majchrzak, T. A., & Grønli, T.-M. (2017). Progressive web apps: The possible web-native unifier for mobile development. In *Proceedings of the 13th International Conference on Web Information Systems and Technologies* (Vol. 2, pp. 344–351). SciTePress. <https://doi.org/10.5220/0006584703440351>
- Choi, H., & Han, S. (2021). An empirical study on factors affecting user acceptance of mobile applications for restaurant ordering. *Journal of Hospitality and Tourism Technology*, 12(3), 514–530. <https://doi.org/10.1108/JHTT-05-2020-0118>
- Flutter. (2024). Flutter documentation: Building layouts. Google. <https://docs.flutter.dev/ui/layout>
- Nawrocki, P., Wrona, K., Marczak, M., & Sniezynski, B. (2021). A comparison of mobile application development approaches. *IEEE Software*, 38(1), 72–80. <https://doi.org/10.1109/MS.2019.2938382>
- Supabase. (2024). Supabase documentation: Database. <https://supabase.com/docs/guides/database>
- Şimşek, İ., & Durmuş, B. (2022). A review of mobile ordering application usage in the food service industry. *Journal of Foodservice Business Research*, 25(4), 432–451. <https://doi.org/10.1080/15378020.2021.1957424>
- Windmill, E. (2020). *Flutter in action*. Manning Publications.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. <https://doi.org/10.1007/978-3-642-29044-2>